Iteration L1: Deploying Locally

In the past, there was a bewildering number of options available for deployment: Ansible, Capistrano, Chef, and Puppet were all popular choices. But starting with Rails 7.1, Docker has become the official default for deployment.

If you're not familiar with Docker images, you can find out more in *Docker for Rails Developers* [Ise19]. Meanwhile, all you need to know is that they're essentially self-contained and portable runtimes that can be deployed by pretty much any cloud-hosting provider. This means you can build and test your deployment locally and then choose your cloud provider later, and even change your mind and move hosts at any time.

You control what goes into a Docker image by creating a Dockerfile. Helpfully, Rails provides a Dockerfile for you to start with. We won't be changing it, so if you're in a hurry, you can skip to *Orchestrate the Deployment* and refer back to this later.

Base Image

We'll look at the Dockerfile in three parts. We'll start with the base image:

rails80/depot_td/Dockerfile

```
# syntax=docker/dockerfile:1
# check=error=true

# This Dockerfile is designed for production, not development. Use with
#   Kamal or build'n'run by hand:
# docker build -t work .
# docker run -d -p 80:80 \
#   -e RAILS_MASTER_KEY=<value from config/master.key> --name work work

# For a containerized dev environment, see Dev Containers:
#   https://guides.rubyonrails.org/getting_started_with_devcontainer.html
```